Autoreason: Autoresearch for Subjective Domains

shl0ms

https://github.com/NousResearch/autoreason

Abstract. We introduce autoreason, a method for iterative LLM refinement in subjective domains where no objective metric exists. Each iteration produces three competing versions of a document—the incumbent, an adversarial revision, and a synthesis—evaluated by a blind judge panel. Fresh isolated agents, Borda count aggregation, and conservative tiebreaking construct a subjective fitness function through the evaluation process itself. With claude-sonnet-4 across 5 task types, autoreason ranked first or second in every 7-judge blind comparison, winning 3/5 outright. Chain-of-thought judging accelerates convergence 3×; Monte Carlo analysis confirms 80% convergence with consistent quality; game-theoretic analysis validates fully transitive preferences.

With a stronger model (Sonnet 4.6), the method fails on open-ended tasks—synthesis consistently outcompetes the incumbent, preventing convergence. Eight remedy experiments identify the root cause as *scope*, not evaluation methodology. On a scope-constrained task (500-word pitch), the same stronger model converges in 10 passes and wins a blind comparison against all baselines—the same baselines that dominated the unconstrained version. Autoreason's effectiveness depends on the interaction between evaluation methodology and task specification: the synthesis operator needs a bounded improvement space to reach a quality ceiling.

1 Introduction

LLMs exhibit systematic failures when iterating on subjective work: sycophancy, overcriticism, overcompromise, authorship bias, scope drift, and context collapse. These failures persist across prompting strategies—even in code generation, iterative refinement leads to progressive degradation [2]. Self-Refine [4] demonstrates that single-agent critique-and-revise loops can improve outputs but provides no mechanism to detect when revisions make things worse. Huang et al. [5] show LLMs cannot reliably self-correct without external feedback, motivating our external judge panel.

We present autoreason, extending Karpathy's autoresearch [1] to subjective domains. Each iteration produces three competing versions—the incumbent (A), an adversarial revision (B), and a synthesis (AB)—evaluated by a blind judge panel. The method constructs a fitness function through the evaluation process itself, analogous to how science uses peer review where math can use proofs.

We tested two modes. **Single pass**: one cycle through critic → B → AB → judge (5 LLM calls). **Convergence loop**: repeats the cycle with a full judge panel until the incumbent survives consecutive challenges. As we show in Section 4.1, the single pass does not outperform simpler baselines—autoreason's value emerges from iteration. We further show that with stronger models, the method's effectiveness depends on task scope: scope constraints restore convergence and quality where unconstrained tasks fail (Section 5).

2 Method

Critic. A fresh agent identifies problems in the current incumbent A. Finds problems only—no fixes. Optionally receives ground-truth data for fact-checking (Section 6).

Author B. A fresh agent revises A based on the critique. Sees the task, A, and the critique. No drafting history.

Synthesizer. A fresh agent receives A and B with randomized labels. Produces AB by taking the strongest elements of each.

Judge Panel. 3 fresh agents rank A, AB, B via Borda count (3/2/1 points). Randomized labels and presentation order. Conservative tiebreak: incumbent wins ties. Convergence at k=2 consecutive A wins.

Every role is a fresh, isolated agent with no shared context. The original task prompt anchors all evaluation. Exact prompts are provided in Appendix A.

2.1 Formal Framing

Let d_t denote the incumbent document at pass t. Each pass produces candidates $\{d_t, B(d_t), S(d_t, B(d_t))\}$ where B is the adversarial revision operator and S is the synthesis operator. A judge panel J defines a stochastic preference relation: $J(x, y) \in [0, 1]$ is the probability that x is ranked above y. The winner is selected by Borda aggregation over n judges:

$$d_{t+1} = \arg\max_{c \in \{A,B,AB\}} \sum_{i=1}^{n} (3 - r_i(c))$$

where $r_i(c)$ is judge i's rank for candidate c, with ties broken in favor of d_t.

This defines a Markov chain over document states. Convergence ($d_{t+1} = d_t$ for k consecutive passes) corresponds to an absorbing state. By the Condorcet jury theorem, if each judge independently identifies the better document with $p > 0.5$, panel accuracy increases with n. However, same-

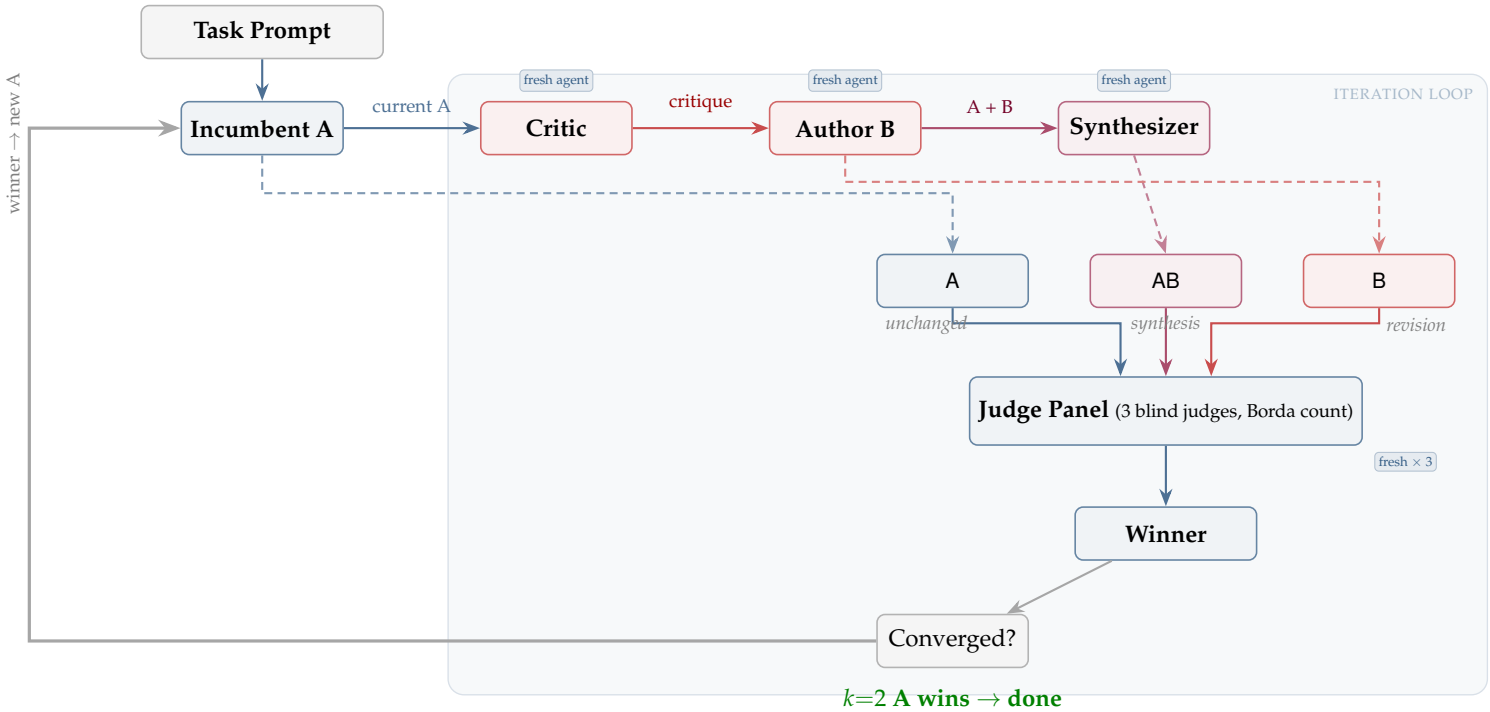

Figure 1. The autoreason loop (Figure 1). Each pass produces three candidates: the unchanged incumbent (A), a synthesis (AB), and an adversarial revision (B). A blind judge panel selects the winner via Borda count. If A survives k=2 consecutive passes, the loop terminates. Every role is a fresh, isolated agent.

model judges share systematic biases, bounding effective independence below n.

The connection to PSRO [10] is direct: each pass adds a candidate to an empirical game. Our retroactive analysis (Section 4.5) validates this framing: preferences are fully transitive across 91 passes (1.1% Condorcet cycle rate), and Elo ratings plateau early, confirming the Markov chain reaches its absorbing state.

3 Experiments

All experiments used claude-sonnet-4-20250514 (temp=0.8 authors, 0.3 judges, max_tokens=4096) unless otherwise noted. Five tasks: (1) go-to-market strategy, (2) notification system design, (3) remote work policy, (4) competitive positioning, (5) incident response. Each task ran autoreason to convergence (k=2) plus four baselines (conservative, improve_this, harsh_critic, critique_and_revise) at 15 passes each from the same initial output, evaluated by a 7-judge blind panel. The paper-writing experiment (Section 6) used claude-opus-4. Scaling experiments (Section 5) used claude-sonnet-4-6.

4 Results

4.1 Single Pass vs. Iteration

Method	Calls	T1	T2	T3	T4	T5
Harsh critic	1	**30**	**31**	23	**30**	**31**
Critique & rev.	1	**30**	30	**31**	28	23
Autoreason	5	24	22	24	17	24
Improve this	1	14	14	18	23	20
Conservative	1	7	8	9	7	7

Table 1. Single-pass Borda scores (max 35). Bold indicates task winner. Autoreason places 3rd at 5× the cost.

Method	T1	T2	T3	T4	T5
Autoreason	**35**	19	**29**	25	**31**
Critique & rev.	13	**22**	26	**32**	19
Harsh critic	18	19	27	20	26
Conservative	21	19	15	18	15
Improve this	18	11	8	10	14

Table 2. Iterative Borda scores (max 35) at 15 passes. Autoreason wins tasks 1, 3, 5; critique-and-revise wins tasks 2, 4. Compare Table 1: iteration inverts the ranking.

Single-pass autoreason averaged 22.2 Borda, placing 3rd on 4/5 tasks (Table 1). With iteration, it won 3 of 5 tasks

(avg 27.8, rank 1.4), outperforming all single-pass methods (Table 2, Figure 2). The judge's protection against bad revisions compounds: each pass either improves the incumbent or leaves it unchanged, while baselines accumulate drift unchecked.

Autoreason excels on tasks with genuine tradeoffs (strategy, policy, incident response). Critique-and-revise wins on tasks with concrete technical requirements (system design, competitive positioning) where a direct find-and-fix loop is more efficient (Figure 3).

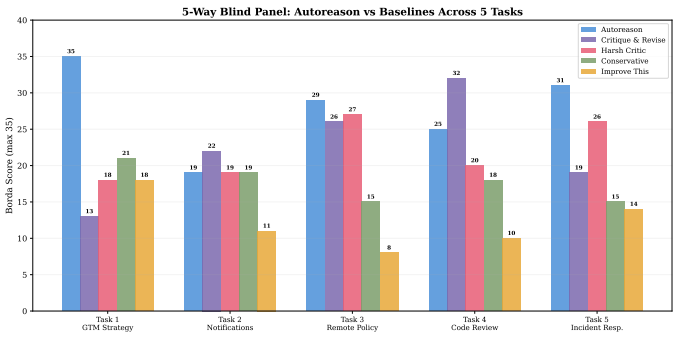

Figure 2. Per-task Borda scores at 15 iterative passes.

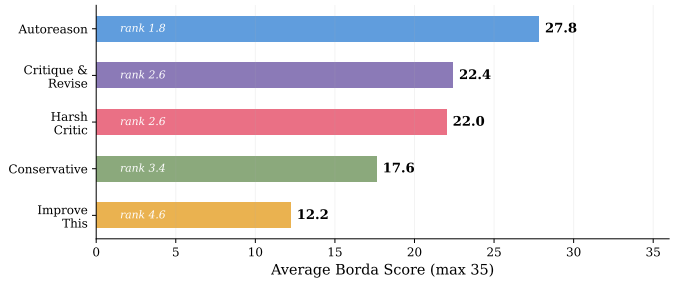

Figure 3. Aggregate performance. Autoreason (27.8 avg Borda, rank 1.4) never placed below 2nd.

4.2 Qualitative Improvement

The initial Task 1 output was a generic startup playbook: \$49/user pricing, \$100K MRR target with a 3-person team, no customer validation. After 14 passes, the converged output contained quantified pain (\$15K/incident $\times$ 6/yr), team-based pricing (\$1,499/mo matching the actual buying motion), validation from 50+ customer interviews with 75% pilot success rate, competitive positioning against named tools (OPA/Gatekeeper, ArgoCD, Flux), and unit economics (CAC \$2K, LTV \$54K, 90% gross margin). The adversarial process killed the unrealistic revenue assumptions.

4.3 Convergence and Trajectory

All 5 tasks converged at threshold k=2, requiring 9–28 passes (Figure 4). Policy tasks converge faster (10 passes) than multi-stakeholder operational processes (28 passes).

Figure 5 shows the Task 1 trajectory. At pass 15, A had won twice consecutively. Eleven more passes followed—a 7-judge panel comparing pass 15 vs pass 25 chose pass 15 by 6–1. Post-convergence passes are noise.

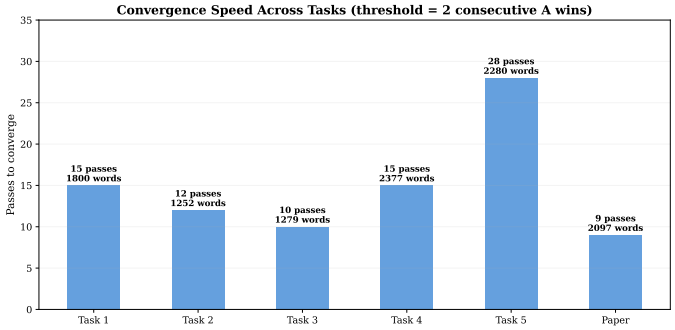

Figure 4. Passes to convergence across all tasks. Task 5 required the most (28); the paper (9) the least.

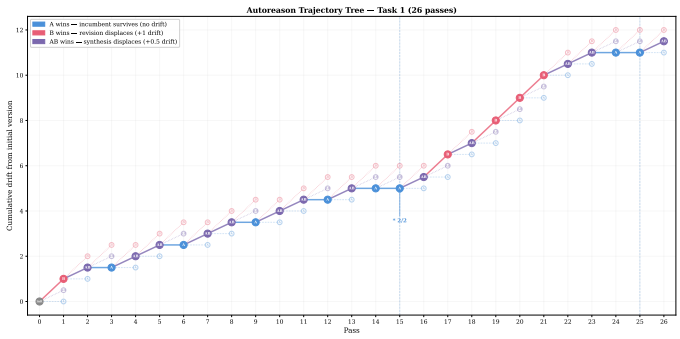

Figure 5. Trajectory tree, Task 1 (26 passes). Stars mark convergence points.

4.4 Chain-of-Thought Judges

Adding "think step by step about each version" to the judge prompt—no architecture changes—dramatically improves convergence (Table 3). CoT judges produce scores of 8–9 for A wins (vs baseline's 6–7), acting as a debiasing mechanism: judges must articulate specific strengths before ranking, preventing verbosity bias.

Judge Variant	Task 1	Task 2
Baseline (holistic)	14–15 passes	12 passes
Chain-of-thought	**5 passes**	**8 passes**
Decomposed (3 specialists)	7 passes	—

Table 3. CoT reduces convergence 3$\times$ on Task 1. All subsequent experiments use CoT judges.

4.5 Robustness

Monte Carlo. Five independent runs of Task 1: 4/5 converged (80%). Convergence pass varied (6, 8, 14, 17) but final word counts clustered tightly (1451–1660), suggesting a consistent quality ceiling regardless of path (Figure 6).

Game-theoretic validation. Across 91 passes: only 1 Condorcet cycle (1.1%—near-perfect transitivity). Elo ratings plateau by pass 5–10. Pairwise dominance is fully transitive: autoreason > critique-and-revise > harsh_critic > conservative > improve_this. No rock-paper-scissors dynamics.

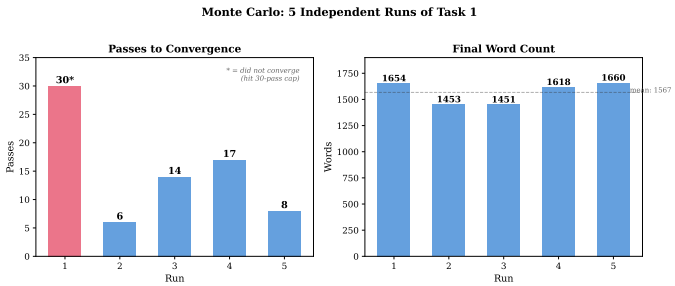

Figure 6. Monte Carlo: 5 runs of Task 1. Convergence speed varies; final output quality does not.

5 Model Scaling

We ran autoreason on Task 2 using claude-sonnet-4-6 (a stronger model) with CoT judges. The initial result was a complete reversal—autoreason came last (Table 4). Two rounds of remedy experiments then identified the root cause and a fix.

Method	Borda	1st Place
Critique & rev.	**31**	3
Improve this	30	4
Harsh critic	23	0
Conservative	14	0
Autoreason	7	0

Table 4. Sonnet 4.6, Task 2 (unconstrained). Autoreason ran 50 passes without converging; all baselines ran 15 passes.

Over 50 passes, A won only 6 times (12%). AB dominated with 30+ wins. The stronger model produced syntheses so consistently preferred by judges that the incumbent could never survive. This is structurally different from the bloat/prune oscillation with Sonnet 4: with 4.6, judges were decisive enough (scores of 8–9 for AB) to never let A through.

5.1 Round 1: Convergence Remedies

We tested four modifications (Sonnet 4.6, Task 2, CoT judges, 50-pass cap):

Remedy	Passes	A	AB	B	Conv.?
Margin (≥2 pts)	15	3	8	4	**Yes**
All combined	17	3	12	2	**Yes**
Scope-aware judges	35	4	25	6	No
Plateau detection	35	2	22	11	No
Baseline	*50*	*6*	*30+*	*—*	*No*

Table 5. Convergence remedies. Only the margin requirement (AB/B must win by ≥2 Borda) produces convergence.

The margin requirement converged at passes 15 and 17 (Figure 7). Scope-aware judges and plateau detection had no effect.

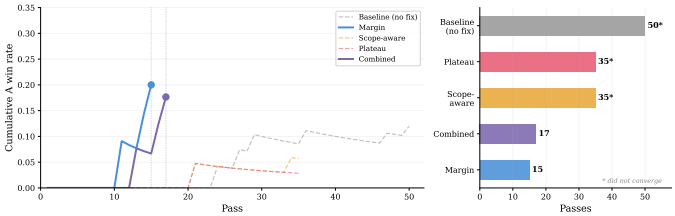

Figure 7. Left: cumulative A win rate (solid = converged). Right: passes to termination (* = cap).

But convergence ≠ quality. A 7-judge panel showed margin-converged output still places 4th–5th (Table 6). The margin rule solves termination but not quality—the incumbent accumulates synthesis drift across 15–17 passes of near-constant displacement.

Method	Borda (/49)	1st
Critique & rev.	**43**	2
Improve this	42	4
Harsh critic	39	1
Autoreason (combined)	26	0
Autoreason (margin)	23	0
Conservative	12	0
Autoreason (baseline)	11	0

Table 6. Margin recovers convergence but not quality. Autoreason still loses to all active baselines.

5.2 Round 2: Root Cause

Three evaluation-side modifications (anchored judges, subtractive synthesis, both combined) plus a scope-constrained task (500-word startup pitch from 8 fixed facts):

Modification	Passes	A	AB	B	Conv.?
Constrained task	**10**	**4**	**6**	**0**	**Yes**
Anchored judges	25	4	12	9	No
Subtractive synth.	17	0	10	7	No
Anchored + subtractive	17	1	6	10	No

Table 7. Only the constrained task converges. Evaluation-side modifications do not.

The constrained task converged at pass 10 with A scoring 9—the strongest convergence with Sonnet 4.6 in any experiment. Word counts stayed within 480–694 (vs 1895–2700 unconstrained). B never won a single pass.

A 7-judge blind panel confirmed quality (Table 8):

Method	Borda (/35)	1st
Autoreason	**30**	**3**
Improve this	27	2
Conservative	25	2
Harsh critic	13	0
Critique & rev.	10	0

Table 8. Constrained task: autoreason wins with Sonnet 4.6. Compare Table 4: the baselines that won unconstrained now lose—they bloated past the 500-word limit.

The ranking inverts (Figure 8). Critique-and-revise expanded to 932 words; autoreason stayed at 632. The constrained task penalizes exactly the drift that autoreason's architecture prevents.

6 Writing This Paper

We ran autoreason on this paper using claude-opus-4 with 3 Opus judges and ground-truth context (actual experimental data). Converged in 9 passes.

Ground-truth critic. Without ground-truth access, the initial Opus generation hallucinated a fabricated ablation study, fake confidence intervals, wrong model names, and incorrect role descriptions (Figure 9). With ground-truth access, the critic caught all four on pass 1—mirroring peer review, where the reviewer checks supplementary materials.

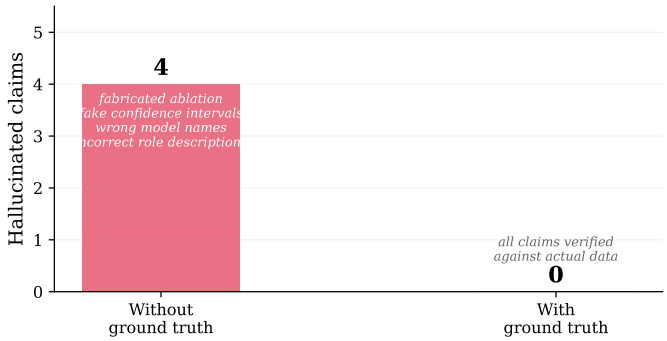

Figure 9. 4 hallucinated claims without ground truth; 0 with it.

Judge panel integrity. An earlier mixed panel (Opus + Sonnet + Gemini) failed to converge for 11+ passes because Gemini's output failed our parser, reducing the panel to 2 judges. Fixed to 3 working judges, the same incumbent converged in 2 passes (Figure 10). A broken judge doesn't add noise—it prevents equilibrium.

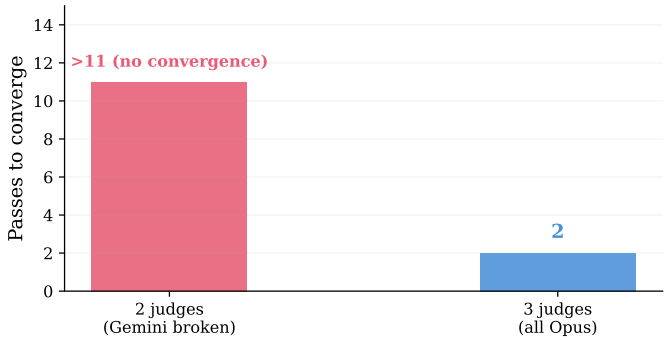

Figure 10. Broken parser: >11 passes, no convergence. 3 working judges: 2 passes.

7 Related Work

Autoresearch [1] uses val_bpb as an objective fitness function; autoreason extends this to subjective domains. **SlopCodeBench** [2] shows iterative code degradation that prompting can't fix. **ACE** [3] documents context collapse in iterative rewriting. **Self-Refine** [4] demonstrates single-agent iterative improvement but Huang et al. [5] show self-correction is unreliable without external feedback. **AI Safety via Debate** [6] grounds adversarial evaluation: truth has a natural advantage when agents argue before a judge. **Constitutional AI** [7] uses principle-based self-evaluation as training signal. **LLM-as-Judge** [8] documents systematic biases motivating our randomization and panel design. **LLM Council** [9] uses multi-model panels for single-turn evaluation; autoreason adds iteration with role separation. **PSRO** [10] provides the game-theoretic framework.

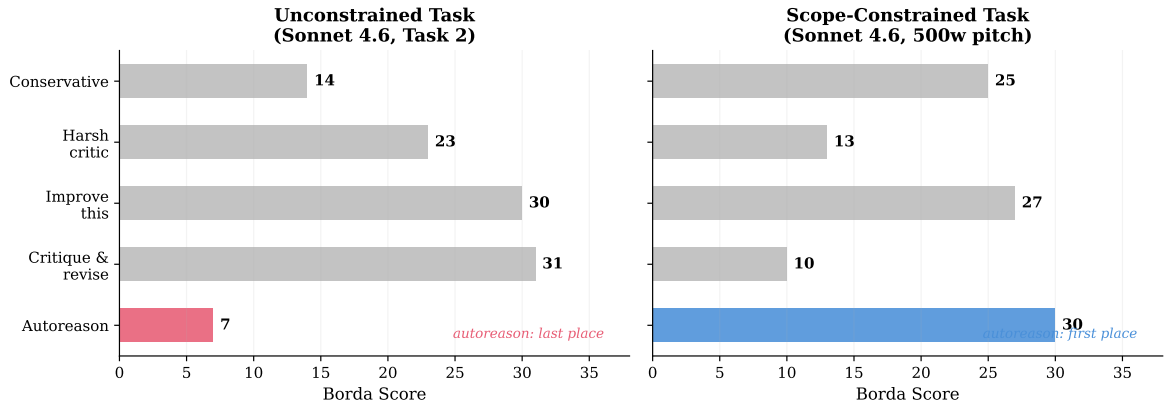

Figure 8. The scaling story. Same model (Sonnet 4.6), same method, opposite results. Left: unconstrained task, autoreason last (red). Right: scope-constrained task, autoreason first (blue).

8 Discussion

Task-dependent advantage. Autoreason is not universally superior. It excels where genuine tradeoffs exist. For tasks with concrete, verifiable requirements, simpler methods suffice. Use autoreason when you don't know which framing will produce the best result; use critique-and-revise when the improvement direction is clear.

CoT judges should be the default. The cheapest improvement: 3× faster convergence, more decisive scores, no architecture changes.

Convergence as stopping rule. The threshold filters noise, not guarantees optimality. Post-convergence passes on Task 1 degraded quality. The Elo analysis confirms plateaus early regardless of whether the threshold is met.

Scope is the central variable. The bloat/prune oscillation with Sonnet 4 (Figure 11) signals underdetermined scope. With stronger models, this oscillation becomes one-directional drift—but scope constraints eliminate it entirely, restoring both convergence and quality (Section 5). The synthesis operator needs a bounded improvement space. When the task constrains scope mechanically, the incumbent can reach a ceiling. When it doesn't, there is always something to add.

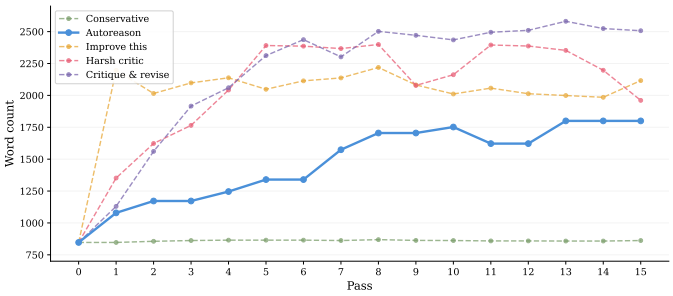

Figure 11. Word count trajectories (15 passes, same start). Autoreason (solid) grows controllably; baselines stagnate or bloat.

Limitations. All results from one model family (Anthropic). No human evaluation. The constrained-task result is from a single task—generalization untested. The paper-writing experiment used Opus, not Sonnet. Scaling remedy experiments each ran once; Monte Carlo replication would strengthen the claims.

9 Conclusion

Autoreason constructs useful fitness functions for subjective domains through systematic isolation and blind evaluation. With Sonnet 4, it ranked first or second across all 5 tasks. With the stronger Sonnet 4.6, it fails on open-ended tasks but wins on scope-constrained ones—beating the same baselines that dominated the unconstrained comparison. Eight remedy experiments confirm that the root cause is scope, not evaluation: the synthesis operator needs a bounded improvement space to reach a quality ceiling.

The practical recommendation: pair autoreason with explicit scope constraints (word limits, required sections, fixed deliverables), especially as models improve. The broader insight: the obstacle to iterative LLM self-improvement is not model capability but the interaction between evaluation methodology and task specification. As models get more capable, the task specification must get more precise—otherwise the improvement space is unbounded and every revision looks locally better while the trajectory drifts globally.

References

[1] A. Karpathy. Autoresearch, 2024.

[2] G. Orlanski et al. SlopCodeBench: Benchmarking how coding agents degrade over iterative tasks. *arXiv:2603.24755*, 2026.

[3] J. Zhang et al. ACE: Agentic context engineering. *ICLR*, 2026.

[4] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. *NeurIPS*, 2023. *arXiv:2303.17651*.

[5] J. Huang et al. Large language models cannot self-correct reasoning yet without external feedback. *arXiv:2310.01798*, 2023.

[6] G. Irving, P. Christiano, D. Amodei. AI safety via debate. *arXiv:1805.00899*, 2018.

[7] Y. Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv:2212.08073*, 2022.

[8] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023. *arXiv:2306.05685*.

[9] A. Karpathy. LLM Council, 2025.

[10] M. Lanctot et al. A unified game-theoretic approach to multiagent reinforcement learning. *NeurIPS*, 2017.

[11] K. Arrow. *Social Choice and Individual Values*. Wiley, 1951.

A Prompts

A.1 Critic

```
System: You are a critical reviewer. Your only job is to find real problems. Be specific
and concrete. Do not suggest fixes.
User: Find real problems with this proposal. Focus on: things that won't work as
described, complexity that doesn't pay for itself, assumptions that are wrong, missing
pieces. Do NOT propose fixes. Just the problems.
```

A.2 Author B

```
System: You are a senior consultant revising a proposal based on specific criticisms.
Address each valid criticism directly. Do not make changes not motivated by an identified
problem.
User: [TASK] + [VERSION A] + [CRITIC OUTPUT] Revise to address these problems. For each
change, state which problem it fixes.
```

A.3 Synthesizer

```
System: You are given two versions as equal inputs. Take the strongest elements from each
and produce a coherent synthesis. This is not a compromise.
User: [TASK] + [VERSION X] + [VERSION Y] (labels randomized)
```

A.4 Judge (Baseline)

```
System: You are an independent evaluator. You have no authorship stake in any version.
User: [TASK] + Three proposals (randomized labels and order). Rank best to worst.
RANKING: [best], [second], [worst]
```

A.5 Judge (Chain-of-Thought)

```
System: You are an independent evaluator. Think carefully before deciding.
User: [TASK] + Three proposals. For each, think step by step: 1. What does it get
right? 2. What does it get wrong or miss? 3. Are numbers and claims defensible? 4. Is
detail appropriate or bloated? After reasoning, rank all three. RANKING: [best], [second],
[worst]
```

B Trajectory Trees

Figure 12 shows trajectory trees for all experiments.

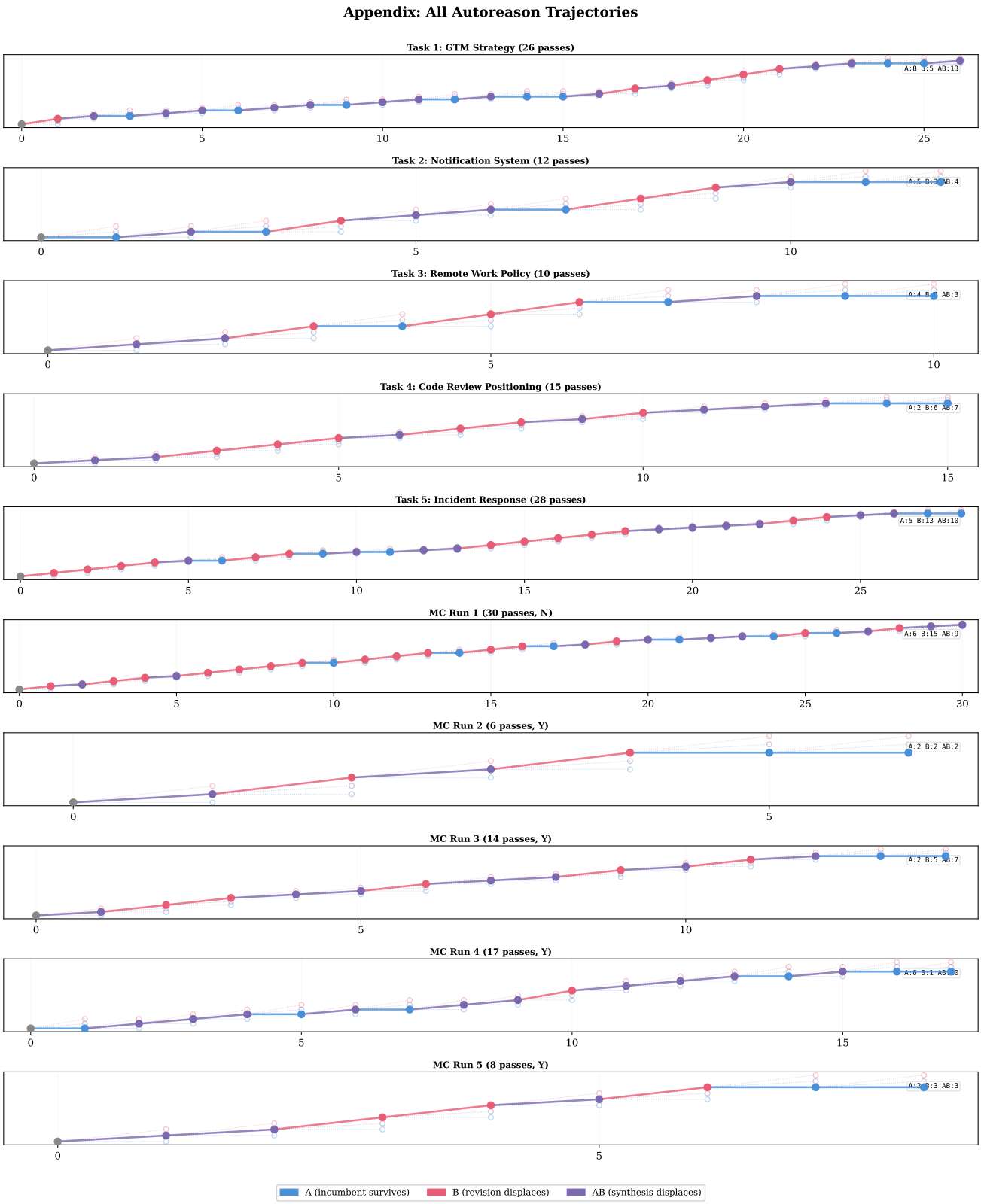

Figure 12. All trajectories. Tasks 2–3 converge quickly (10–12 passes). Task 5 requires 28 passes. Monte Carlo runs show variable convergence speed (6–30 passes) on the same task.